CONTROL DE VERSIONES

1. ¿QUÉ ES EL CONTROL DE VERSIONES?

El control de versiones es un sistema que permite registrar los cambios realizados sobre uno o varios archivos a lo largo del tiempo. Su principal objetivo es conservar un historial que permita conocer cómo ha evolucionado un proyecto y recuperar versiones anteriores cuando sea necesario.

Por ejemplo, una persona puede estar trabajando en un documento y realizar diferentes modificaciones durante varios días. Sin un sistema de control de versiones, tendría que guardar manualmente diferentes copias del archivo, como:

- documento_final.docx
- documento_final_2.docx
- documento_final_definitivo.docx
- documento_final_definitivo_2.docx

Con un sistema de control de versiones, los cambios quedan registrados de manera organizada. Esto permite:

- Consultar versiones anteriores.
- Comparar cambios.
- Identificar cuándo se realizó una modificación.
- Saber quién realizó determinados cambios.
- Recuperar información eliminada.
- Mantener un historial del proyecto.

Por estas razones, el control de versiones es una herramienta útil tanto para proyectos individuales como para trabajos colaborativos.

2. SISTEMAS DE CONTROL DE VERSIONES LOCALES

Los primeros métodos utilizados para controlar versiones consistían principalmente en realizar copias de los archivos y almacenarlas en diferentes carpetas.

Aunque este método era sencillo, tenía varias dificultades. El usuario podía confundirse entre las diferentes copias, modificar un archivo equivocado o eliminar accidentalmente una versión importante.

3. ¿QUÉ ES GIT?

Git es un sistema de control de versiones distribuido diseñado para registrar y administrar los cambios realizados en un proyecto.

A diferencia de un sistema que depende completamente de un servidor central, Git mantiene en el computador local una gran parte de la información necesaria para trabajar con el proyecto.

Esto permite realizar acciones como:

- Registrar cambios.
- Consultar el historial.
- Comparar versiones.
- Crear ramas.
- Recuperar versiones anteriores.
- Preparar modificaciones.
- Confirmar cambios.

Git también utiliza una estructura basada en repositorios. Un repositorio contiene los archivos del proyecto y la información necesaria para llevar el seguimiento de sus diferentes versiones.

4.VENTAJAS DE GIT

- Historial de cambios. Git permite conservar un historial de los cambios realizados. Esto facilita conocer cómo ha evolucionado un proyecto
- Trabajo local. Muchas operaciones pueden realizarse directamente en el computador sin necesidad de estar conectado permanentemente a Internet.
- Recuperación de información. Al conservar diferentes estados del proyecto, Git facilita recuperar información cuando se comete un error.
- Trabajo colaborativo. Git permite que diferentes personas trabajen sobre un mismo proyecto y posteriormente compartan sus modificaciones.
- Uso de ramas. Las ramas permiten desarrollar diferentes funcionalidades o realizar modificaciones sin afectar inmediatamente la versión principal del proyecto.
- Organización. El historial de commits permite mantener una descripción organizada de las modificaciones realizadas.

5. GIT Y GITHUB

Git es el sistema de control de versiones que se instala y utiliza en el computador.

GitHub es una plataforma que permite alojar repositorios Git de manera remota y facilita la colaboración, el almacenamiento y el intercambio de proyectos.

Por ejemplo, una persona puede utilizar Git en su computador para realizar cambios en un proyecto y posteriormente utilizar GitHub para almacenar una copia remota del repositorio.

6. CONCEPTOS BÁSICOS DE GIT

- Repositorio. Es el espacio donde se almacenan los archivos del proyecto y la información relacionada con su historial.
- Commit. Es un registro de los cambios realizados en el proyecto. Cada commit debe tener un mensaje que permita identificar qué modificación se realizó.
- Rama o Branch. Es una línea independiente de desarrollo. Permite realizar cambios sin modificar directamente otra rama.
- Clonar. Consiste en crear una copia local de un repositorio remoto.
- El comando utilizado es:. git clone URL_DEL_REPOSITORIO
- Push. Permite enviar los cambios registrados localmente hacia un repositorio remoto. git push
- Pull. Permite obtener cambios existentes en un repositorio remoto. git pull

7. IMPORTANCIA DE LA LÍNEA DE COMANDOS

Git puede utilizarse mediante diferentes herramientas gráficas, pero la línea de comandos permite acceder directamente a sus principales funciones.

Algunos comandos importantes para comenzar son:

git --version

git status

git add

git commit

git push

git pull

git clone

Aprender estos comandos es importante porque permite comprender qué está sucediendo con el repositorio y facilita posteriormente el uso de herramientas gráficas.

La documentación oficial de Pro Git utiliza la línea de comandos como una de las principales formas de aprender y trabajar con Git.